
Documentation: network_monitor.py

Overall Purpose: The Network Port Guard

This module's job is to act as a security guard for the server's network "doors" (known as **ports**). It specifically watches the ports you've listed in config.json to see if any programs are "listening" on them for incoming connections.

For the Juice Shop project, this is crucial for two main reasons:

1. **Service Availability:** We want to ensure that the Juice Shop application (or any other critical service) is correctly listening on its designated port (e.g., port 3000). If it stops, this monitor will tell us.
2. **Security:** If a malicious program manages to get onto the server, it might open a new port to listen for commands from an attacker (this is often called a "backdoor"). This monitor will spot any unexpected listeners on the ports we care about.

The Key Tool: The psutil Library

To get information about network connections, this monitor relies on a powerful third-party Python library called **psutil** (which stands for "process and system utilities").

Think of psutil as a universal toolkit that allows our Python script to ask the operating system for detailed information about what's happening under the hood. Instead of writing complex code for Linux, another for Windows, etc., we can just ask psutil, and it handles the details for us. In this monitor, we use it to get a complete list of all active network connections.

The Logic: Comparing Snapshots in Time

This monitor doesn't watch things in real-time like the watchdog monitors. Instead, it works by taking a "snapshot" of the network state every few seconds and comparing it to the previous

snapshot to see what has changed.

Analogy: Guard Shift Change

- Imagine a security guard (run method) starting their shift. Their first job is to take a new photo (current_conns) of all the open, listening doors.
 - They then compare this new photo to the photo left by the guard from the previous shift (self.reported_conns).
 - **"What new doors are open now that weren't open before?"** These are the new_conns. The guard reports them.
 - **"What doors were open before that are now closed?"** These are the closed_conns. The guard reports these too.
 - At the end of their shift, they leave their new photo (current_conns) for the next guard to use as the "previous" record. This is the most important step for the cycle to continue.
-

Code Breakdown

NetworkMonitor Class

`__init__(self, ...)` (The Constructor)

- `super().__init__(...)`: As always, this runs the standard setup from the BaseMonitor blueprint.
- `self.reported_conns = set()`: This is the monitor's **memory**. It creates an empty set to store the connections it saw during its last check. A set is a data type that stores a collection of unique items, making comparisons very fast.

`run(self)`

This is the main method that is executed on a loop by the EDR agent. It contains the entire logic for one "guard shift."

1. **Get the Current Picture:**

- It gets the list of listen_ports from the config file.
- It creates a new empty set called current_conns to store the snapshot for this run.
- It calls psutil.net_connections(), which gives it a list of every network connection on the system.
- It loops through this list, filtering for only the connections that are in a 'LISTEN' state and are on one of the ports we're monitoring.
- For each match, it adds a unique identifier (port, pid) to the current_conns set.

2. **Compare to the Previous Picture:**

- new_conns = current_conns - self.reported_conns: This simple line uses a set operation. It subtracts the old set from the new set, and the result is everything that is in current_conns but was NOT in self.reported_conns. For each new connection, it logs a NETWORK-LISTEN-START alert.
- closed_conns = self.reported_conns - current_conns: This does the reverse. It finds everything that was in the old set but is now missing from the new set. For each closed connection, it logs a NETWORK-LISTEN-STOP alert.

3. **Update the Memory:**

- self.reported_conns = current_conns: This is the final, crucial step. The "snapshot" from the current run is saved as the official record, overwriting the old one. This prepares the monitor for the next check.

4. **Error Handling (try...except):**

- The code is wrapped in a try...except block to handle potential problems.
- psutil.AccessDenied: This is a common error. On many systems, you need administrator (root) privileges to view all network connections. If the agent is run without enough permissions, this error will occur.
- **Smart Disabling:** If an AccessDenied error occurs, the monitor logs a warning and then **disables itself** by setting self.monitor_config['enabled'] = False. This is very clever because it prevents the monitor from trying and failing on every single loop, which would flood your logs with thousands of identical error messages.